

08 Geometric Transformations

8.3 Rotation

Rotation matrices: orthogonal, determinant 1, commuting successive rotations — from the plane to 3D axis rotation

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>



Core Skills You Will Master

1

Matrix multiplication implements rotation about the origin: $\mathbf{y}=\mathbf{R}(\vartheta)\mathbf{x}$, with θ the counterclockwise angle

2

The rotation matrix $\mathbf{R}(\vartheta)$ is orthogonal: $\mathbf{R}^{-1}=\mathbf{R}^T$, $\mathbf{R}@\mathbf{R}^T=\mathbf{R}^T@ \mathbf{R}=\mathbf{I}$, determinant always 1

3

The Gram matrix $\mathbf{R}^T\mathbf{R}$ and its relation to inner products and L2 norms of the columns

4

Successive planar rotations commute:
 $\mathbf{R}(\beta)\mathbf{R}(\alpha)=\mathbf{R}(\alpha)\mathbf{R}(\beta)=\mathbf{R}(\alpha+\beta)$

5

Swapping the order of rotation and scaling changes the result — matrix multiplication generally does not commute

6

Matrix multiplication implements 3D axis rotation: independent rotation about x_1, x_2, x_3 , following the right-hand rule

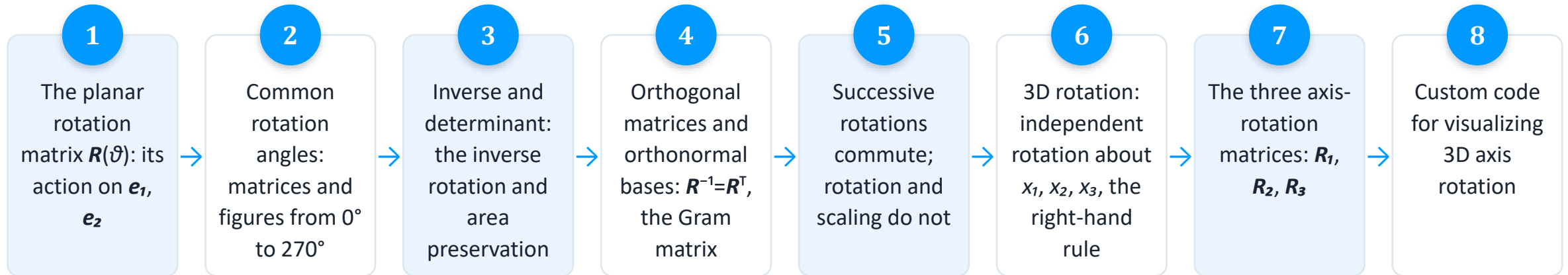
7

3D rotation matrices are also orthogonal with determinant 1, verifiable via Laplace expansion

8

In 3D, rotations about different axes do not commute; only rotations about the same axis commute

From Planar Rotation to 3D Axis Rotation



Seven Lenses for Every Transform in This Chapter

① Matrix-multiplication view: how the matrix acts on $\mathbf{0}$, $\mathbf{e}_1$, $\mathbf{e}_2$, and the all-ones vector $\mathbf{1}$

② The determinant: is $|\det|$ greater than 1? Positive, negative, or zero? Determinants of special matrices

③ How the determinant reflects the area or volume change caused by the transform

④ The inverse matrix (does it exist?) and the inverse transform; inverses of special matrices

⑤ Linear-combination view of the parallelogram/parallelepiped grid, and whether dimension drops

⑥ Basis view: does the transform matrix form a basis, and what kind of basis

⑦ Properties of the transform matrix: symmetric, diagonal, orthogonal, invertible, commutative

The Transform Matrix $R(\theta)$ for a Counterclockwise Rotation by θ

$$R(\theta) = [\cos\theta, -\sin\theta; \sin\theta, \cos\theta]$$

- Rotation transforms are used widely in machine learning, computer graphics, robotics, and physics; later chapters on spectral decomposition, PCA, and SVD all rely on rotation
- Rotation is usually carried out via matrix multiplication; this section covers planar rotation first, then 3D rotation

Vector $\mathbf{x}$ Rotated Counterclockwise by θ Becomes $\mathbf{y}$

$$[y_1, y_2]^T = \mathbf{R}(\theta)[x_1, x_2]^T$$

- Given a column vector $\mathbf{x}$ in the plane, rotating it counterclockwise by ϑ about the origin gives a new vector $\mathbf{y}$
- Rotation never changes a vector's length, only its direction — the key feature distinguishing it from scaling

Figure 1 · R Acting on e_1, e_2

- $Re_1 = [\cos\vartheta, \sin\vartheta]^T$ extracts the first column of R ; $Re_2 = [-\sin\vartheta, \cos\vartheta]^T$ extracts the second column

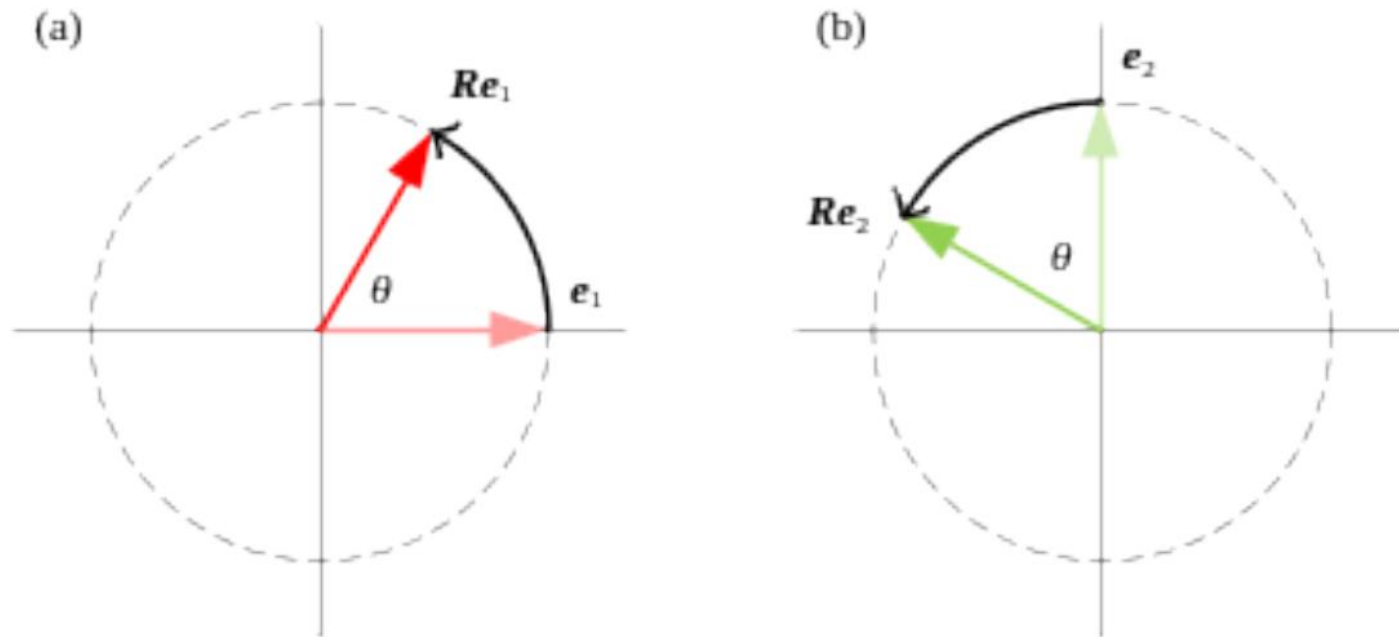


图 1. R 对 e_1 、 e_2 的作用

Re_1 and Re_2 Are e_1, e_2 After Rotating by θ

$$\mathbf{Re}_1 = [\cos\theta, \sin\theta]^T \quad \mathbf{Re}_2 = [-\sin\theta, \cos\theta]^T$$

- As shown in Figure 1(a), $[\cos\vartheta, \sin\vartheta]^T$ is where $\mathbf{e}_1$ lands after a counterclockwise rotation by ϑ
- As shown in Figure 1(b), $[-\sin\vartheta, \cos\vartheta]^T$ is where $\mathbf{e}_2$ lands after a counterclockwise rotation by ϑ

Figure 2 · Two Examples of Rotation

- (a) a rotation of $2\pi/3$ (120°); (b) a rotation of $-3\pi/4$ (-135° , i.e. 135° clockwise)

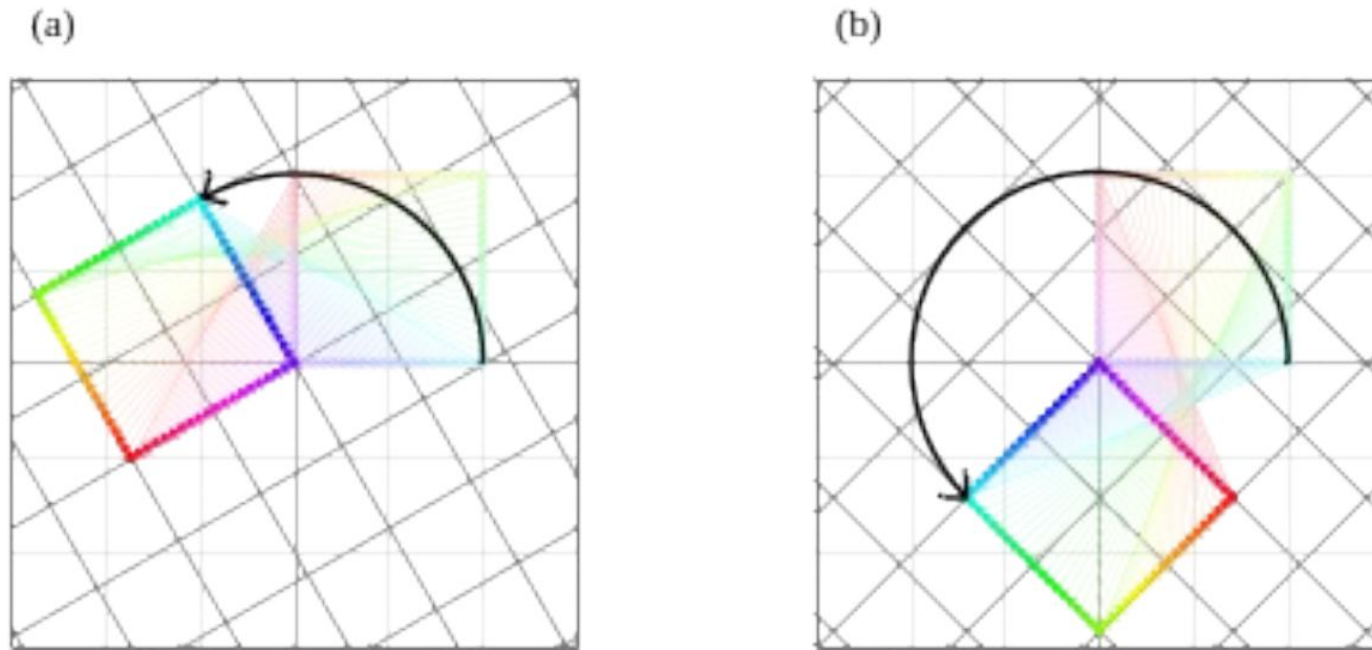
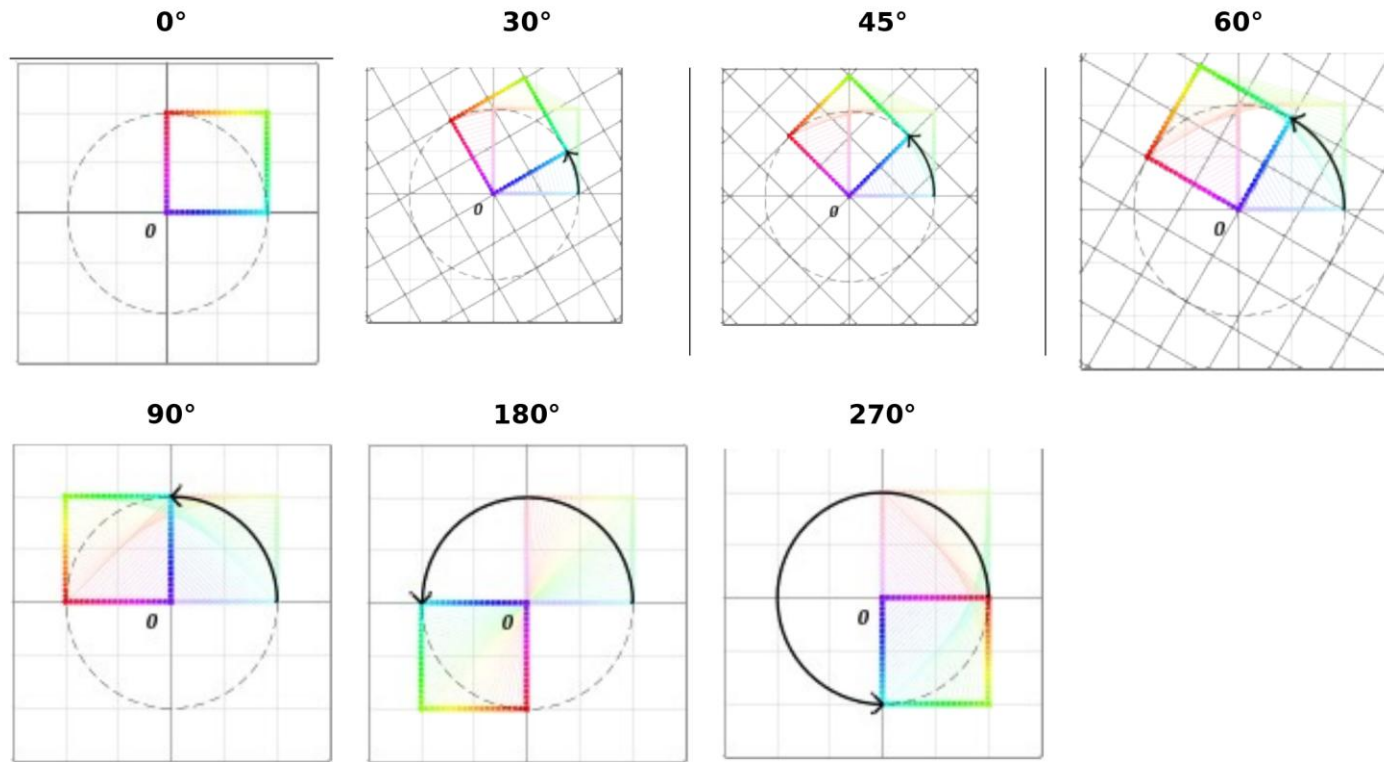


图 2. 旋转的两个例子

Table 1 · Several Common Rotation Angles

- 0° is the identity I — no change; the 90° , 180° , 270° matrices have entries only 0, 1, -1 , making them easy to compute



Clockwise Rotation by θ Corresponds to Counterclockwise by $-\theta$

$$\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta) = [\cos\theta, \sin\theta; -\sin\theta, \cos\theta]$$

- Inverting $\mathbf{R}(\vartheta)$ gives a matrix that represents a clockwise rotation about the origin by ϑ
- This also tells us that the inverse of a rotation is itself a rotation, just in the opposite direction

Figure 3 · Inverse of a Rotation

- Rotating counterclockwise with R , then clockwise by the same angle with R^{-1} , returns the figure to its original position

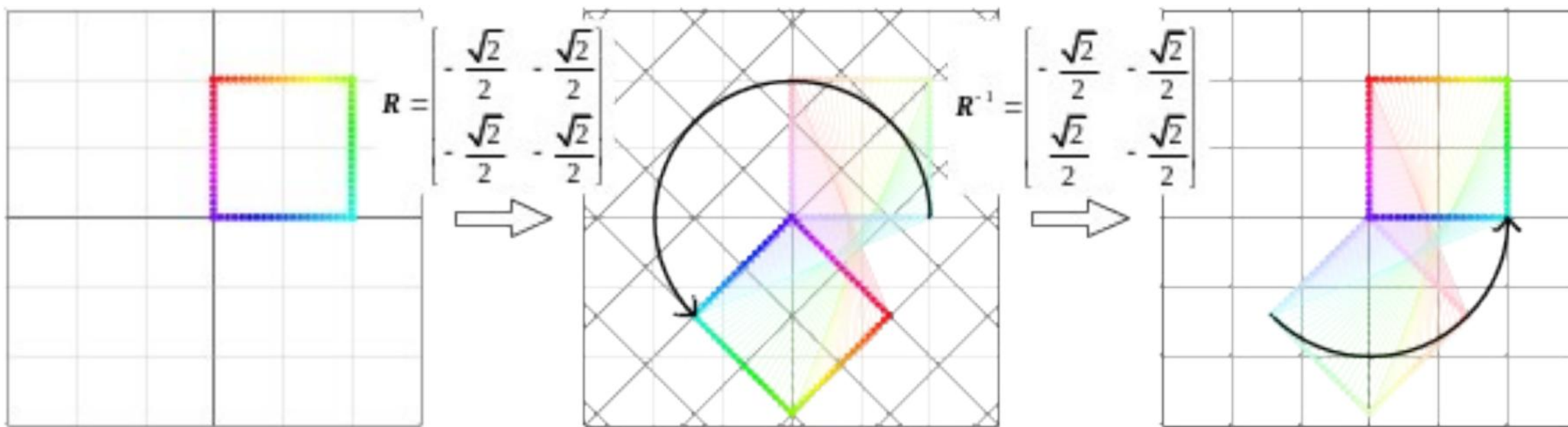


图 3. 旋转的逆运算

The Determinant of a 2D Rotation Matrix Is Always 1

$$\det(R(\theta)) = \cos^2\theta + \sin^2\theta = 1$$

- This shows planar rotation never changes the area of a figure; rotation about the origin also never changes the order of vectors
- A determinant of exactly 1 is a hallmark that distinguishes rotation from scaling, reflection, and other transforms

The Inverse R^{-1} Equals the Transpose of the Rotation Matrix

$$R^{-1} = R^T \quad R @ R^T = R^T @ R = I$$

- Comparing $R^{-1}(\vartheta)$ with the transpose of $R(\vartheta)$, we find the two are identical
- This means R is an orthogonal matrix — the inverse of an orthogonal matrix equals its transpose

The Columns of $R=[r_1, r_2]$ Form an Orthonormal Basis

$$\|r_1\|_2^2 = r_1^T @ r_1 = 1 \quad \|r_2\|_2^2 = r_2^T @ r_2 = 1$$

- Every column of R is a unit vector, so its squared L2 norm is 1; the two columns are also orthogonal: $r_2^T @ r_1 = r_1^T @ r_2 = 0$
- So $R=[r_1, r_2]$ is an orthonormal basis; when $\vartheta=0$, $R=I$, giving the standard basis

R Transposed Times R Is Called the Gram Matrix of R

$$R^T R = [r_1^T @ r_1, r_1^T @ r_2; r_2^T @ r_1, r_2^T @ r_2] = I$$

- The diagonal of the Gram matrix holds each column's self inner product, equal to its squared L2 norm; off-diagonal entries are pairwise inner products
- Later chapters cover the Gram matrix in depth — it is a key tool for understanding orthogonality and independence of vector sets

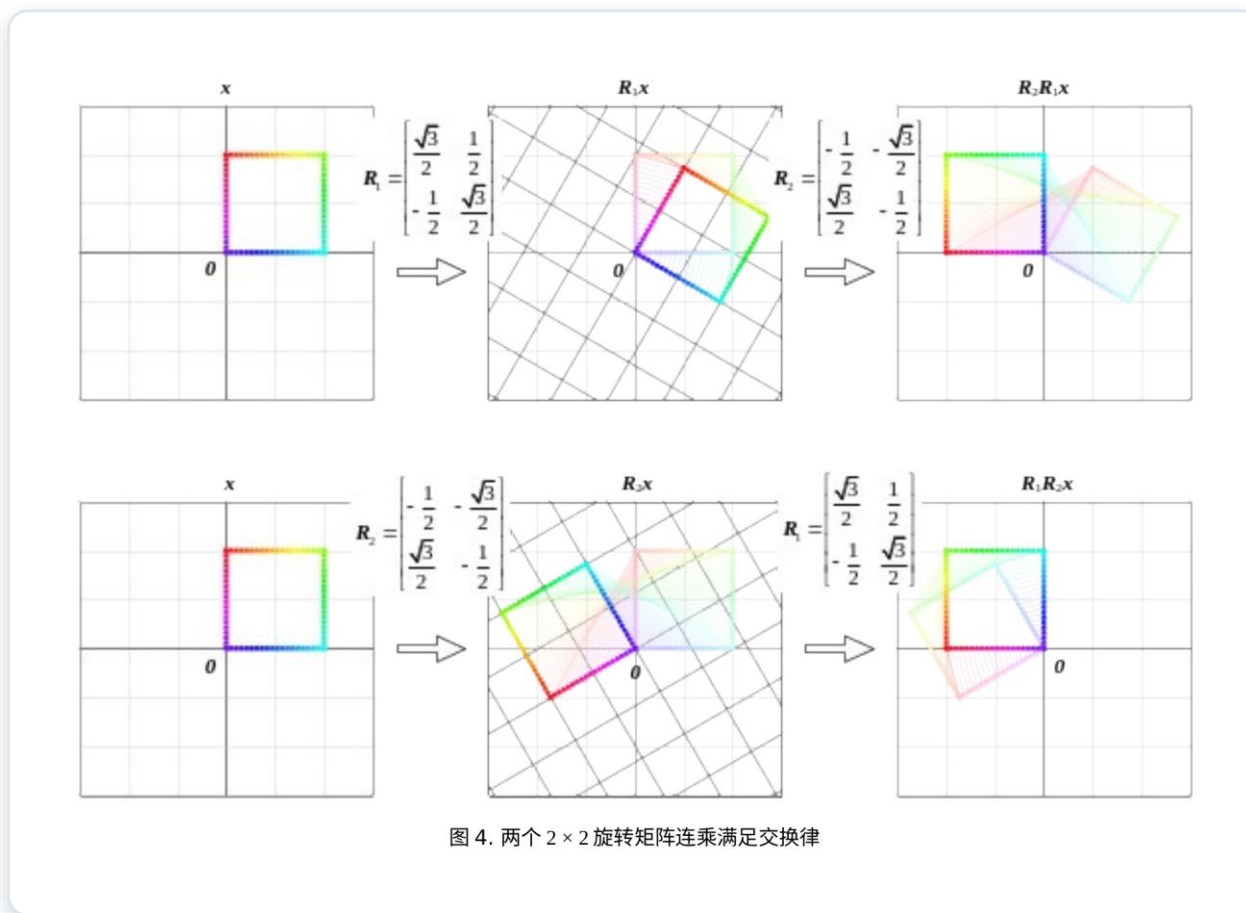
Rotate by α , Then by β : the Total Matrix Is $R(\alpha+\beta)$

$$R(\beta)R(\alpha) = R(\alpha)R(\beta) = R(\alpha+\beta)$$

- Matrix multiplication generally does not commute, but two successive rotations about the origin are a special case: they combine into a single rotation
- This shows planar rotation about the origin is additive — several rotations equal one rotation by the sum of their angles

Figure 4 · Two 2×2 Rotation Matrices Commute

- Top row: R_1 then R_2 . Bottom row: R_2 then R_1 . Both orders produce exactly the same final result



Scale-Then-Rotate $\neq$ Rotate-Then-Scale

$$RS \neq SR$$

- Matrix multiplication generally does not commute, i.e. $AB \neq BA$; composing rotation and scaling is a typical example
- Recall: for $ABCx$, the operations act on x right to left — C , then B , then A

Figure 5 · Scale First, Then Rotate

- $S = \text{diag}(1/2, 2)$ first squashes and stretches the square, then R rotates the result by 60° , giving a tilted parallelogram

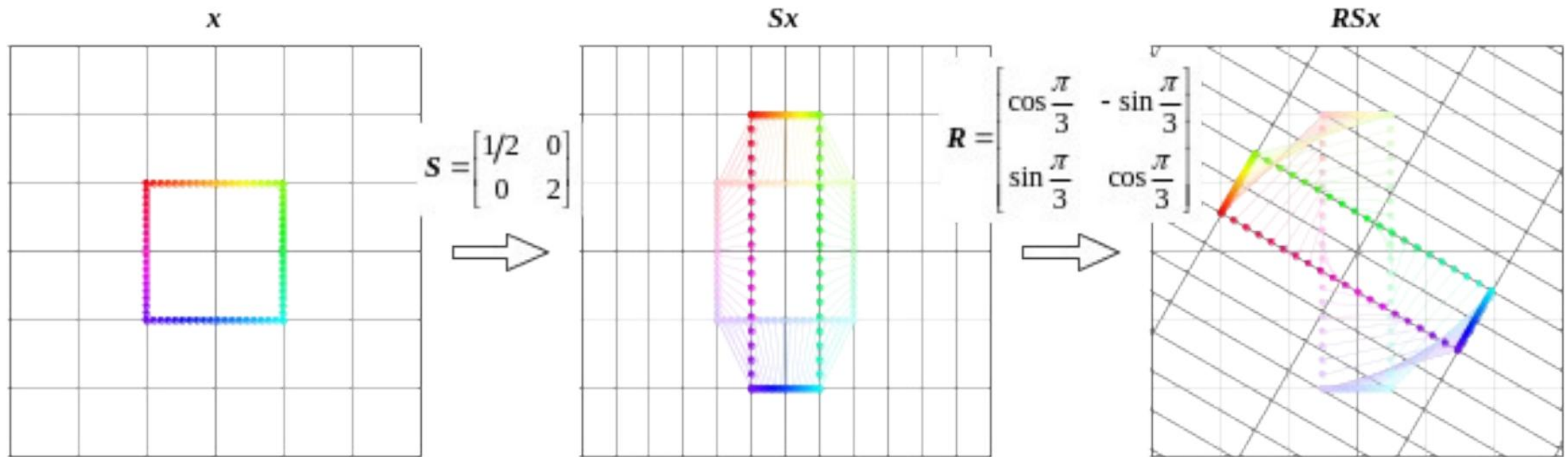


图 5. 先缩放，再旋转

Figure 6 · Rotate First, Then Scale

- R first rotates the square by 60° , then $S=\text{diag}(1/2, 2)$ squashes and stretches it — the final shape clearly differs from Figure 5

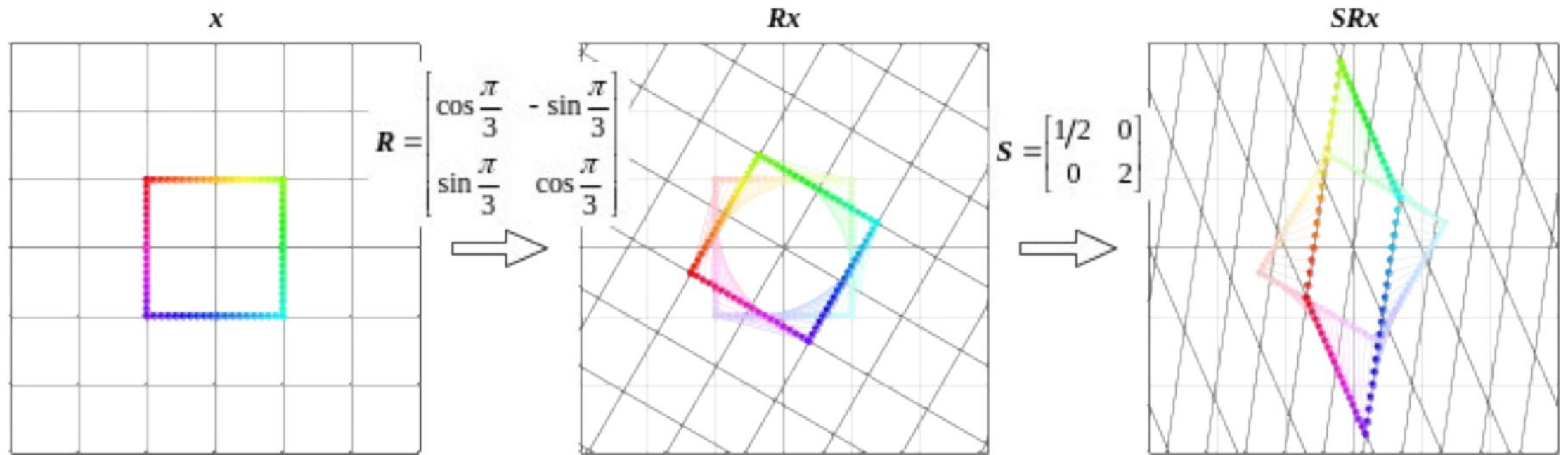


图 6. 先旋转，再缩放

The Transform Matrix R_1 for Rotating by θ_1 About x_1

$$R_1 = [1, 0, 0; 0, \cos\theta_1, -\sin\theta_1; 0, \sin\theta_1, \cos\theta_1]$$

- In 3D, a solid can rotate independently about the x_1, x_2, x_3 axes
- The sign of rotation follows the right-hand rule: thumb along the positive axis, curled fingers show the positive (counterclockwise) direction

Figure 7 · A Solid Rotating Independently About Three Axes

- The unit cube can rotate independently about x_1, x_2, x_3 by angles $\vartheta_1, \vartheta_2, \vartheta_3$

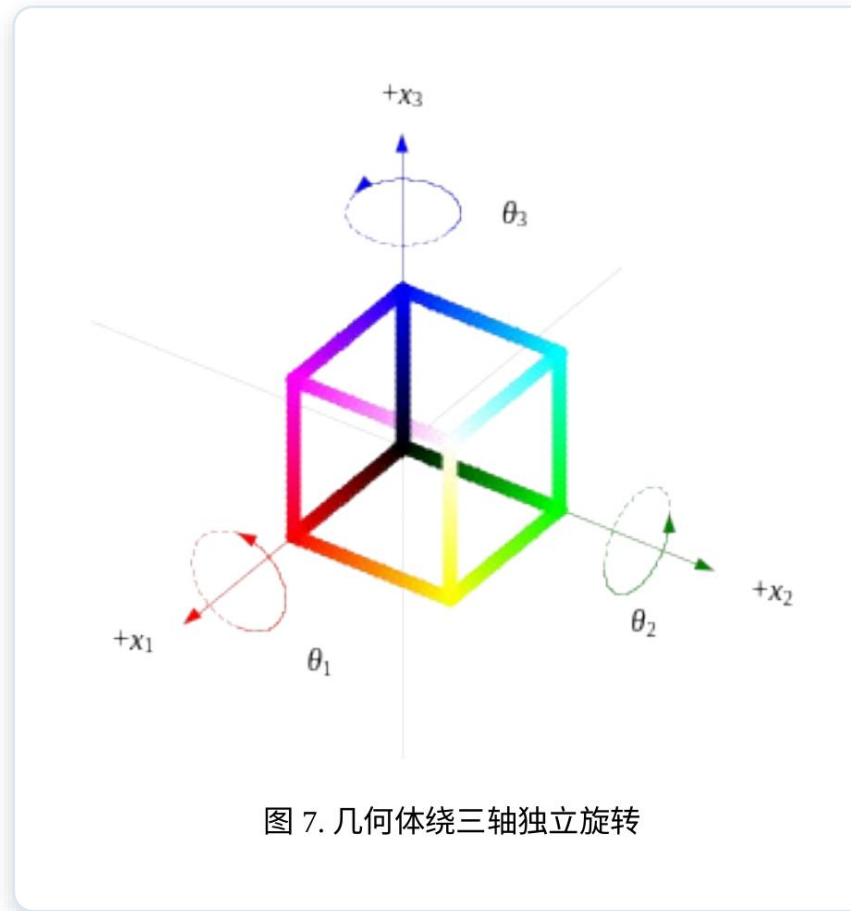


Figure 8 · Three Rotation Angles

- Thumb along the positive axis; the curled fingers show the positive (counterclockwise) direction — the same rule applies to all three axes

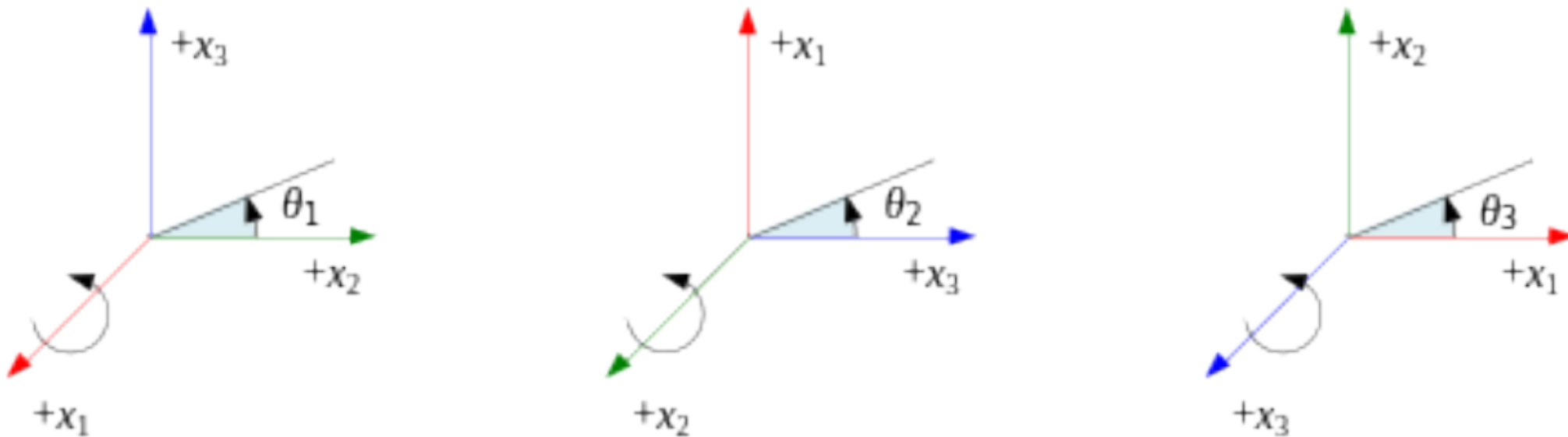


图 8. 三个旋转角度

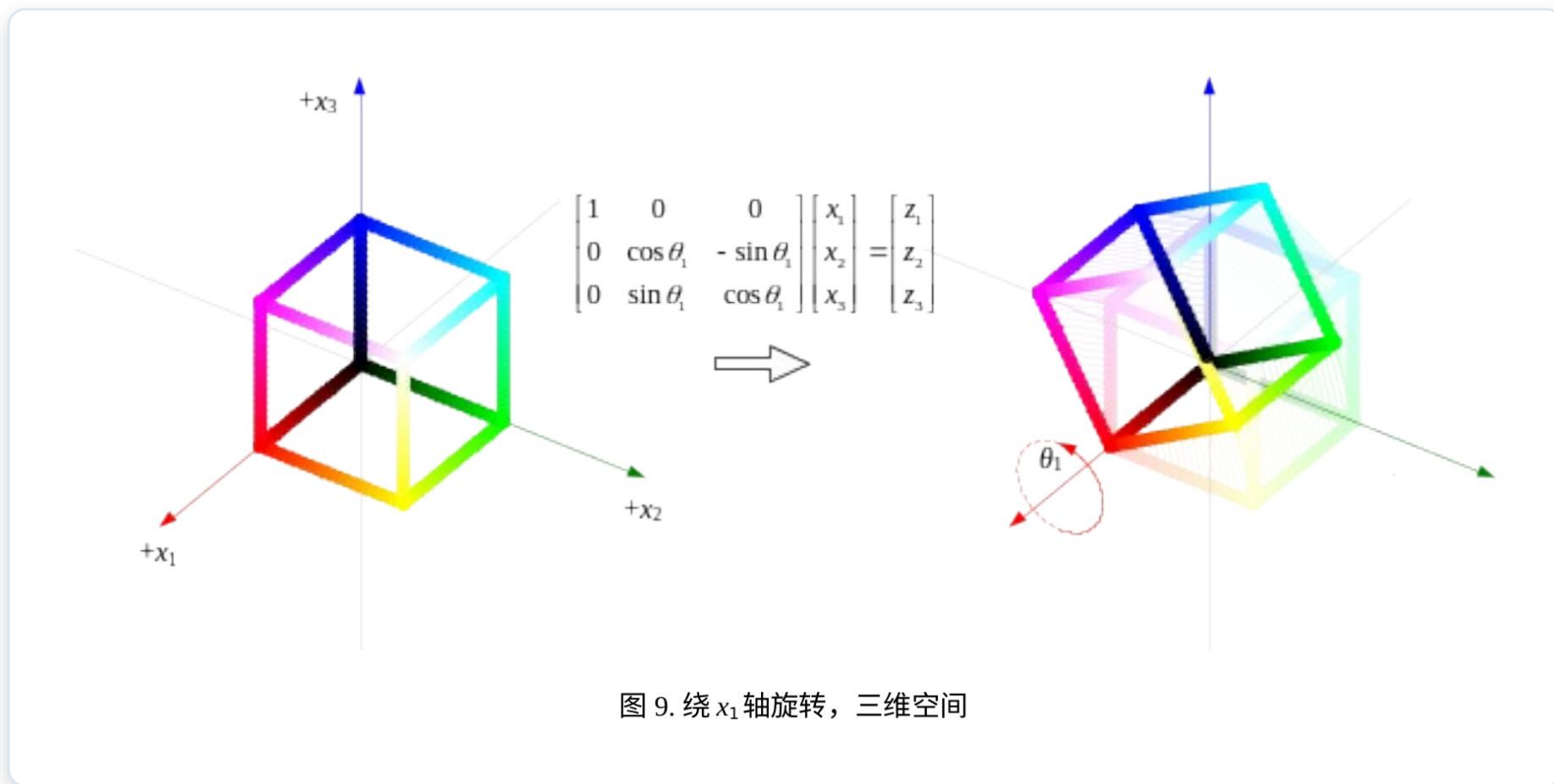
Vector $\mathbf{x}$ Rotated by θ_1 About x_1 Becomes $\mathbf{z}$

$$z_1 = x_1 \quad z_2 = x_2 \cos \theta_1 - x_3 \sin \theta_1 \quad z_3 = x_2 \sin \theta_1 + x_3 \cos \theta_1$$

- $\mathbf{R}_1$ only changes the x_2, x_3 coordinates; x_1 stays fixed — this is exactly what “rotating about x_1 ” means
- Expanding $\mathbf{R}_1 \mathbf{x} = \mathbf{z}$ gives the three equations above, matching the 2D rotation form

Figure 9 · Rotation About the x_1 Axis, 3D Space

- After rotating the unit cube by ϑ_1 about x_1 , the four edges in the x_2x_3 plane turn while the edges along x_1 stay fixed



R_1 Is Also Orthogonal, With Determinant 1

$$R_1^{-1} = R_1^T \quad \det(R_1) = 1$$

- Computing the Gram matrix $R_1^T R_1 = R_1 R_1^T = I$ shows R_1 is orthogonal, an orthonormal basis in 3D
- Using Laplace expansion along the first row (it has the most zeros, minimizing computation) gives $\det(R_1)=1$ — volume is unchanged

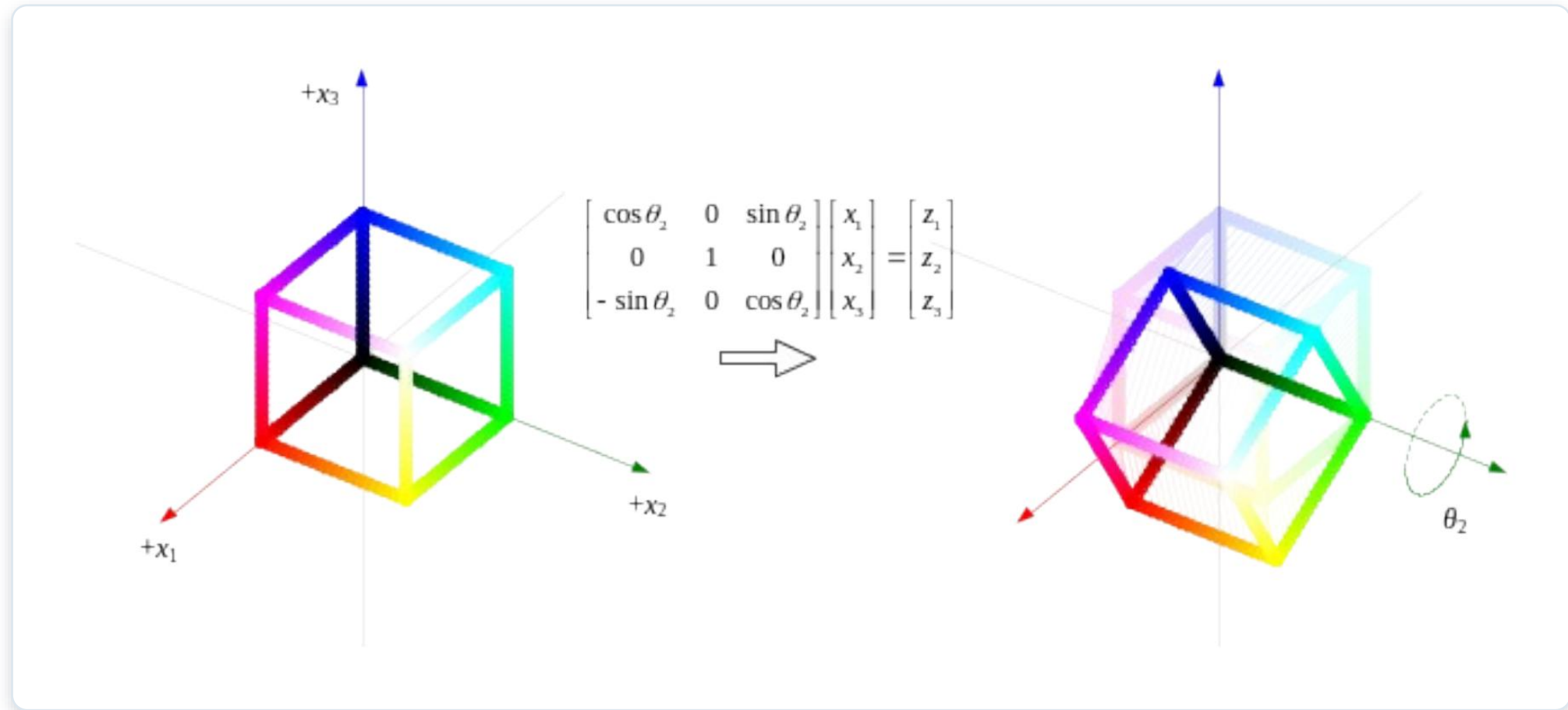
The Transform Matrix R_2 for Rotating by θ_2 About x_2

$$R_2 = [\cos\theta_2, 0, \sin\theta_2; 0, 1, 0; -\sin\theta_2, 0, \cos\theta_2]$$

- R_2 only changes the x_1, x_3 coordinates; x_2 stays fixed
- You can likewise verify R_2 is orthogonal with determinant 1 — try computing its inverse yourself

Figure 10 · Rotation About the x_2 Axis, 3D Space

- After rotating the unit cube by ϑ_2 about x_2 , the edges in the x_1x_3 plane turn while the edges along x_2 stay fixed



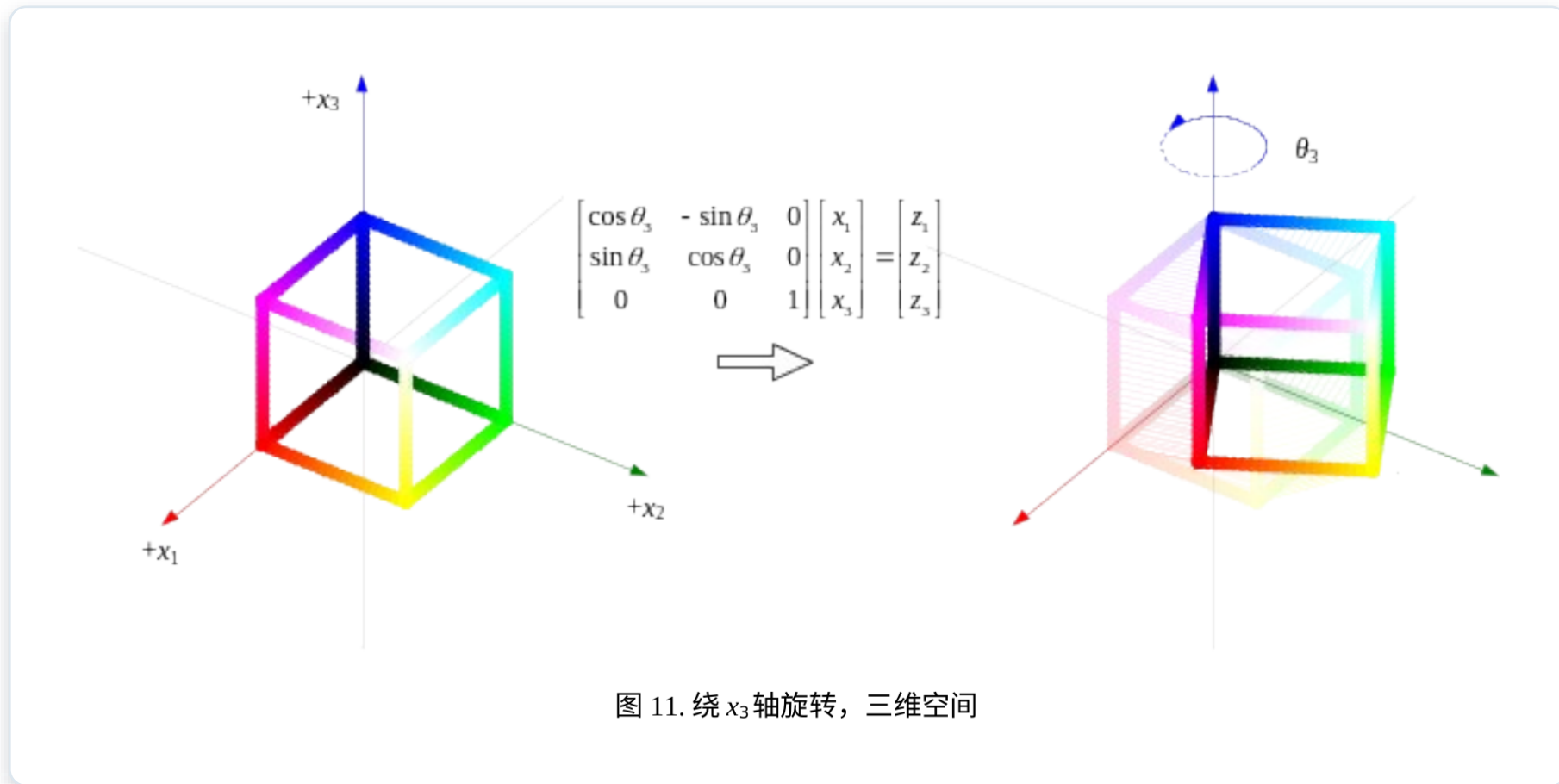
The Transform Matrix R_3 for Rotating by θ_3 About x_3

$$R_3 = [\cos\theta_3, -\sin\theta_3, 0; \sin\theta_3, \cos\theta_3, 0; 0, 0, 1]$$

- R_3 only changes the x_1, x_2 coordinates; x_3 stays fixed, matching the 2D rotation form
- You can likewise verify R_3 is orthogonal with determinant 1 — try computing its inverse yourself

Figure 11 · Rotation About the x_3 Axis, 3D Space

- After rotating the unit cube by ϑ_3 about x_3 , the edges in the x_1x_2 plane turn while the edges along x_3 stay fixed



In 3D, Rotations About Different Axes Do Not Commute

$$R_1 R_2 \neq R_2 R_1 \quad R_1(\theta_\alpha) R_1(\theta_\beta) = R_1(\theta_\beta) R_1(\theta_\alpha)$$

- Rotations about different axes cannot be swapped in order; only rotations about the same axis (different angles) commute, similar to planar rotation
- This differs from planar rotation, where any two rotations commute — in 3D, composition order genuinely affects the final orientation

Custom Visualization

Function: Drawing a Cube

plot_cube(): draws the original and transformed cube

- The original cube's vertex data is assigned to colors; the unit cube maps to RGB space, each vertex's coordinates being its color code
- The edges list enumerates the cube's 12 edges, each given by two vertex indices — 8 points combine into 12 edges
- A for loop draws all edges of the original cube (gray dotted) and the transformed cube (black solid)
- scatter() draws the vertices of the original and transformed cube separately, each vertex in its own color

```
## 可视化函数
def plot_cube(ax, original_cube, transformed_cube):
    colors = original_cube

    # 定义每个卦限的框线顶点对
    edges = [[0, 1], [1, 2], [2, 3], [3, 0],
             [0, 4], [1, 5], [2, 6], [3, 7],
             [4, 5], [5, 6], [6, 7], [7, 4]]

    # 绘制原始立方体框线
    for edge in edges:
        ax.plot(*zip(*original_cube[edge]),
                color="gray", linestyle="dotted")

    # 绘制变换后的立方体框线
    for edge in edges:
        ax.plot(*zip(*transformed_cube[edge]),
                color="black", linestyle="-")

    # 绘制顶点
    ax.scatter(original_cube[:,0], original_cube[:,1],
              original_cube[:,2],
              facecolor=colors, s=50, marker='o', edgecolor='k')

    ax.scatter(transformed_cube[:,0], transformed_cube[:,1],
              transformed_cube[:,2],
              facecolor=colors, s=50, marker='o', edgecolor='k')

    ax.set_xlim(-2, 2); ax.set_ylim(-2, 2); ax.set_zlim(-2, 2)
    ax.axis(False); ax.set_box_aspect([1, 1, 1])
    ax.view_init(elev=30, azimuth=30)
    ax.set_proj_type('ortho')
    ax.plot((-2,2),(0,0),(0,0),color = 'k')
    ax.plot((0,0),(-2,2),(0,0),color = 'k')
    ax.plot((0,0),(0,0),(-2,2),color = 'k')
```

3D Axis Rotation Examples

Rotating the unit cube about the x_1 , x_2 , and x_3 axes

- Defines the 8 vertices of the unit cube;
theta_1=np.pi/6, i.e. 30 degrees
- Builds the R_x1 rotation matrix, applies
cube @ R_x1.T to rotate about x1, then calls
plot_cube() to visualize
- Builds R_x2, R_x3 the same way, rotating
about x2 and x3, reusing the custom
function from Code 1
- All three blocks share the same structure,
differing only in the rotation matrix and axis
— a clean example of function reuse

```

a  ## 定义单位立方体的 8 个顶点坐标
cube = np.array([[0, 0, 0], [1, 0, 0], [1, 1, 0], [0, 1, 0],
                 [0, 0, 1], [1, 0, 1], [1, 1, 1], [0, 1, 1]])

b  ## 绕 x1 轴旋转
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111, projection='3d')

theta_1 = np.pi / 6
R_x1 = np.array([[1, 0, 0],
                 [0, np.cos(theta_1), -np.sin(theta_1)],
                 [0, np.sin(theta_1), np.cos(theta_1)]])

rotated_cube = cube @ R_x1.T
plot_cube(ax, cube, rotated_cube)

c  ## 绕 x2 轴旋转
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111, projection='3d')

theta_2 = np.pi / 6
R_x2 = np.array([[np.cos(theta_2), 0, np.sin(theta_2)],
                 [0, 1, 0],
                 [-np.sin(theta_2), 0, np.cos(theta_2)]])

rotated_cube = cube @ R_x2.T
plot_cube(ax, cube, rotated_cube)

d  ## 绕 x3 轴旋转
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(111, projection='3d')

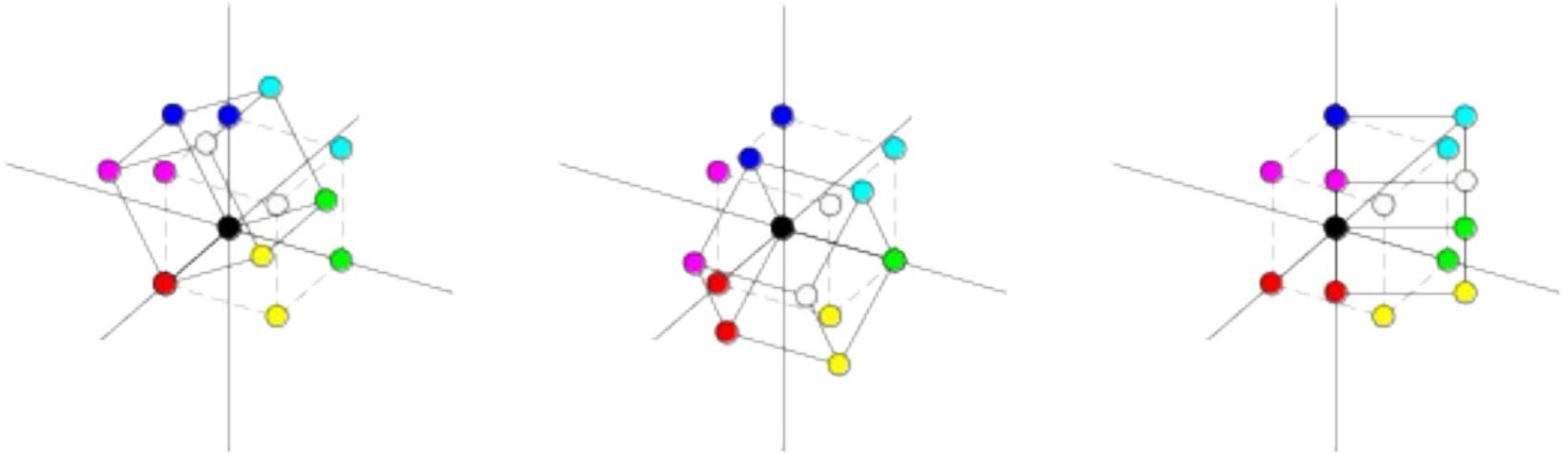
theta_3 = np.pi / 6
R_x3 = np.array([[np.cos(theta_3), -np.sin(theta_3), 0],
                 [np.sin(theta_3), np.cos(theta_3), 0],
                 [0, 0, 1]])

rotated_cube = cube @ R_x3.T
plot_cube(ax, cube, rotated_cube)

```

Figure 12 · 3D Axis Rotation Examples

- Left to right: rotating 30° about x_1 , x_2 , x_3 ; gray dotted lines are the original cube, black solid lines the rotated cube



SUMMARY

What Have We Seen From the Matrix View of Rotation?

Rotation matrix $\mathbf{R}(\vartheta)$: orthogonal, $\mathbf{R}^{-1}=\mathbf{R}^T$, determinant always 1, area/volume unchanged

Successive rotations commute: $\mathbf{R}(\alpha)\mathbf{R}(\beta)=\mathbf{R}(\alpha+\beta)$; but rotation and scaling do not commute

3D rotation: independent about x_1, x_2, x_3 , right-hand rule; same-axis commutes, cross-axis does not

Gram matrix $\mathbf{R}^T\mathbf{R}=\mathbf{I}$: columns are unit length and mutually orthogonal, forming an orthonormal basis

Key Terms in This Section

Rotation Transform 旋转变换 —— a linear transform that turns a figure about the origin or an axis

Orthogonal Matrix 正交矩阵 —— a square matrix satisfying $R^{-1}=R^T$

Standard Basis 标准正交基 —— the orthonormal basis when $\vartheta=0$, i.e. the columns of I

Commutative Law 交换律 —— successive rotations satisfy $R(\alpha)R(\beta)=R(\beta)R(\alpha)$

Laplace Expansion 拉普拉斯展开 —— computing a determinant by expanding along a row or column

Rotation Matrix 旋转矩阵 —— $R(\vartheta)$, orthogonal, determinant 1

Orthonormal Basis 规范正交基 —— columns of unit length that are pairwise orthogonal

Gram Matrix 格拉姆矩阵 —— $R^T R$, diagonal holds squared L2 norms, off-diagonal holds inner products

Right-Hand Rule 右手定则 —— the convention for judging the positive direction of 3D rotation

Gram Matrix Property 格拉姆矩阵性质 —— the Gram matrix of an orthogonal matrix is always the identity

FURTHER LEARNING

Extended Reading

Recap · 8.2 Scaling

Earlier in this chapter — diagonal matrices and scaling transforms in detail

Further Reading · 8.5 Orthogonal Projection

Later in this chapter — orthogonal projection, Gram-Schmidt, QR decomposition

Further Reading · 8.6 Reflection

Later in this chapter — reflection transforms, another case of orthogonal matrices

Open-Source Repository

All code and figures for the book: <https://github.com/Visualize-ML>

Q1 – Q9

Q1 Using matrix multiplication in Python, rotate point (3, 4) counterclockwise by 90° about the origin.

Q3 For planar rotation, rotate 30° then 60° : compute each rotation matrix, then their composite.

Q5 Given points=[(1,0),(0,1),(-1,0),(0,-1),(0,0)], rotate them 45° counterclockwise and print the results.

Q7 Compare: rotating 90° about x_1 then x_2 , versus 90° about x_2 then x_1 — are the results the same?

Q9 Learn what a unit quaternion is, and how it can perform 3D rotation.

Q2 Prove that the inverse of a 2D rotation matrix is its transpose.

Q4 Write a Python function `rotate_point(x1, x2, theta)` that returns the rotated coordinates.

Q6 Use matplotlib to draw a unit square, then rotate it 60° counterclockwise and draw the result.

Q8 Learn what Rodrigues' rotation formula is.

Q10 – Q17

Q10 Learn what the gimbal lock problem is.

Q12 Write a Python function `is_rotation(R)` that checks whether a given 2×2 matrix is a rotation matrix.

Q14 Build $R(30^\circ)$ and $S = \text{diag}(2, 0.5)$; use Python to compare RS and SR — are they equal?

Q16 Verify $R(\theta)$ times $R(-\theta)$ gives the identity, and visualize rotating out and back to the original position.

Q11 Learn about complex numbers, and their connection to rotation.

Q13 Use numpy to verify: for any angle θ , $\det(R(\theta))=1$ and $R(\theta)@R(\theta).T$ equals the identity matrix.

Q15 Write a Python function `compose_rotations(*thetas)` that returns the composite of several planar rotations.

Q17 For the 30° and 60° matrices in Table 1, compute the Gram matrix $R^T R$ and check it equals the identity.

Q18 – Q25

Q18 Write a Python function `rotate_x1(x1,x2,x3,theta)` that returns 3D coordinates rotated about x_1 .

Q20 Verify: two rotations about the same axis, $R_1(\theta_a)$ and $R_1(\theta_b)$, commute; about different axes, they do not.

Q22 Extend the `plot_cube()` function from Code 1 with parameters for cube transparency and edge color.

Q24 Comprehensive: compose $R_1(30^\circ)$, $R_2(45^\circ)$, $R_3(60^\circ)$ in sequence and visualize the final rotated cube.

Q19 Use numpy to verify: with R_1 , R_2 rotating 90° about x_1 , x_2 , compare $R_1 @ R_2$ with $R_2 @ R_1$.

Q21 By hand, use Laplace expansion to compute the determinants of R_2 and R_3 , verifying both equal 1.

Q23 Write a Python function `rotation_matrix_3d(axis, theta)` returning the matrix for axis in `'x1', 'x2', 'x3'`.

Q25 Preview thought: the next section covers projection (determinant 0) — how does that affect a basis compared to rotation (determinant 1)?

10 Prompts to Explore With AI

1 Explain why a rotation matrix's inverse equals its transpose, and how this relates to the definition of an orthogonal matrix.

2 Prove that two successive rotations about the origin commute: $R(\alpha)R(\beta)=R(\beta)R(\alpha)=R(\alpha+\beta)$.

3 Using Python, demonstrate why scale-then-rotate and rotate-then-scale give different results.

4 Explain what the Gram matrix $R^T R = I$ tells us about the geometric properties of a rotation matrix's columns.

5 Explain why a 2D rotation matrix's determinant is always 1, and what that implies about a figure's area.

10 Prompts to Explore With AI

6 Explain why rotations about different 3D axes don't commute, while rotations about the same axis do.

7 Using Python, verify that the 3D rotation matrices R_1 , R_2 , R_3 are all orthogonal with determinant 1.

8 Explain how the right-hand rule determines the positive direction of a 3D rotation, with a concrete example.

9 Explain the connection between Rodrigues' rotation formula and this section's rotation matrices — why is it better suited for rotation about an arbitrary axis?

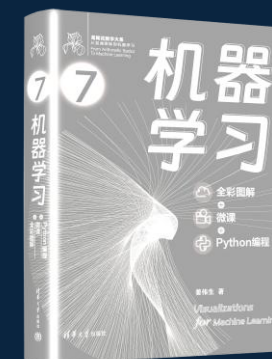
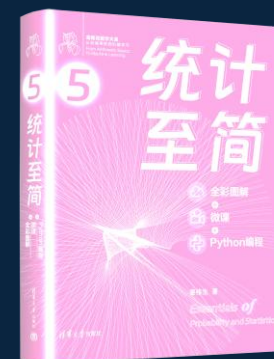
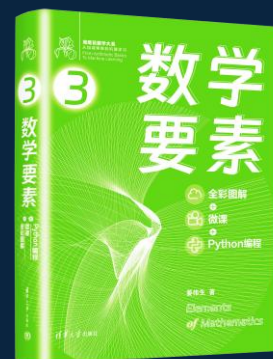
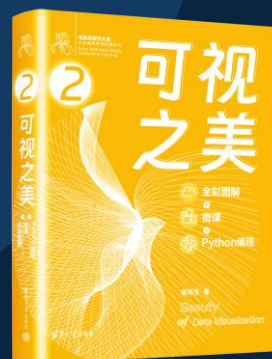
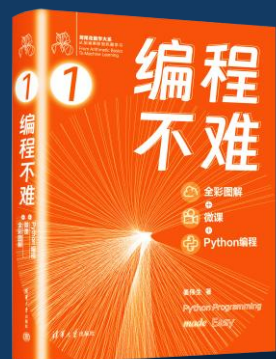
10 Explain why unit quaternions avoid gimbal lock, and what advantages they offer over rotation matrices.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

